Lecture10

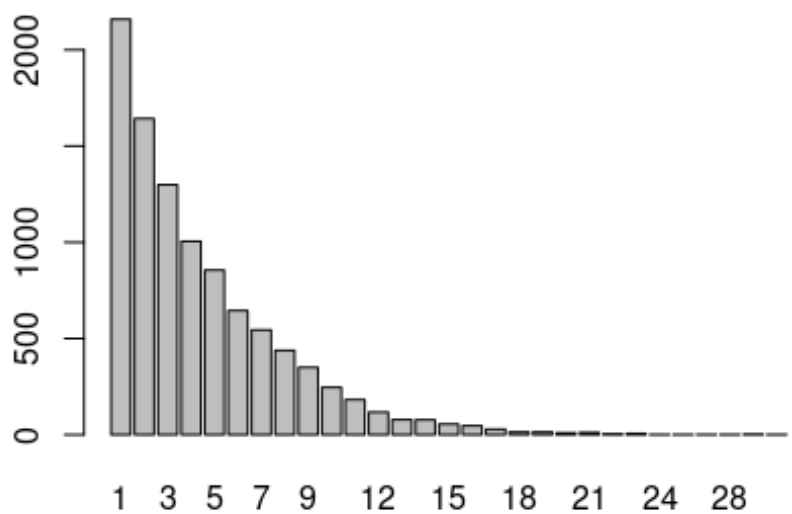
2025-10-30

Association Rules Mining

```
#install.packages("arules")

library("arules")

## Loading required package: Matrix

##
## Attaching package: 'arules'

## The following objects are masked from 'package:base':
##
##     abbreviate, write

data (Groceries)
myGroc <- Groceries # Make a copy for safety
str(myGroc)

## Formal class 'transactions' [package "arules"] with 3 slots
##   ..@ data       :Formal class 'ngCMatrix' [package "Matrix"] with 5 slots
##   .. .. ..@ i       : int [1:43367] 13 60 69 78 14 29 98 24 15 29 ...
##   .. .. ..@ p       : int [1:9836] 0 4 7 8 12 16 21 22 27 28 ...
##   .. .. ..@ Dim     : int [1:2] 169 9835
##   .. .. ..@ Dimnames:List of 2
##   .. .. .. ..$ : NULL
##   .. .. .. ..$ : NULL
##   .. .. ..@ factors : list()
##   ..@ itemInfo   :'data.frame':  169 obs. of  3 variables:
##   .. ..$ labels: chr [1:169] "frankfurter" "sausage" "liver loaf" "ham"
...
##   .. ..$ level2: Factor w/ 55 levels "baby food","bags",..: 44 44 44 44 44
44 44 42 42 41 ...
##   .. ..$ level1: Factor w/ 10 levels "canned food",..: 6 6 6 6 6 6 6 6 6 6
...
##   ..@ itemsetInfo:'data.frame':  0 obs. of  0 variables

summary(myGroc)

## transactions as itemMatrix in sparse format with
##  9835 rows (elements/itemsets/transactions) and
##  169 columns (items) and a density of 0.02609146
##
## most frequent items:
##       whole milk other vegetables       rolls/buns             soda
##             2513             1903             1809             1715
##           yogurt          (Other)
```

```
##             1372            34055
##
## element (itemset/transaction) length distribution:
## sizes
##    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15
16
## 2159 1643 1299 1005  855  645  545  438  350  246  182  117   78   77   55
46
##   17   18   19   20   21   22   23   24   26   27   28   29   32
##   29   14   14    9   11    4    6    1    1    1    1    3    1
##
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   2.000   3.000   4.409   6.000  32.000
##
## includes extended item information - examples:
##        labels  level2           level1
## 1 frankfurter sausage meat and sausage
## 2     sausage sausage meat and sausage
## 3  liver loaf sausage meat and sausage
```

```
barplot(summary(myGroc)@lengths)
```



Explanation:

- **myGroc** is a giant list of every single customer receipt (your sales data).

- **summary(myGroc)** goes through every receipt and counts how many items are in it (e.g., Receipt 1 has 3 items, Receipt 2 has 5 items, etc.). The result of that summary is a large object that contains many different statistics.
- The ***@lengths*** part is a special way in R to specifically extract (pull out) just the piece of information that lists: "How many receipts had 1 item, how many had 2 items, how many had 3 items," and so on.

Exploring the Data

```
myGrocIF <- itemFrequency(myGroc)
myGrocIF <- sort(myGrocIF)
```

Explanation:

1. **myGrocIF <- itemFrequency(myGroc)**

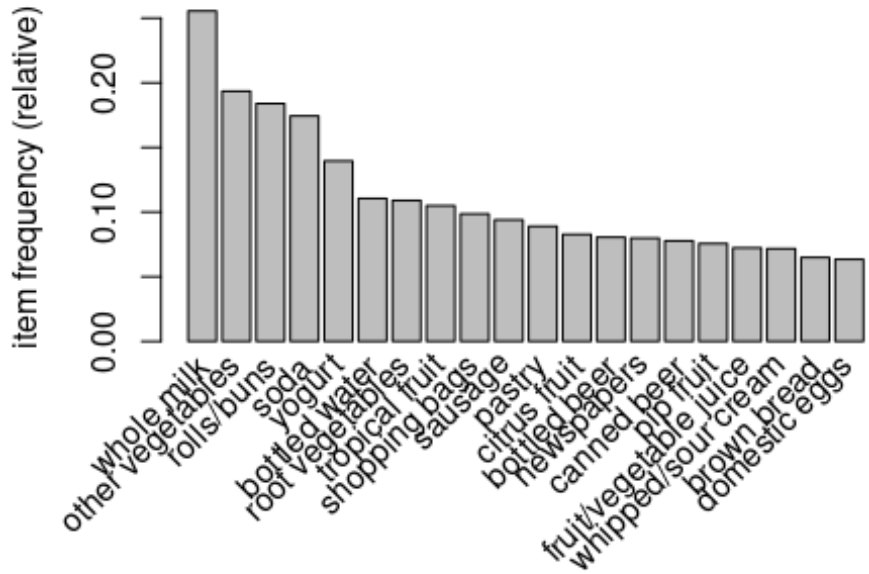
This line tells the computer: "Go through every single receipt and count how often each item (like 'milk' or 'bread') appears. Then, figure out what percentage of all receipts include that item."

2. **myGrocIF <- sort(myGrocIF)**

This line tells the computer:

"Take that list you just made and put it in order from the least popular item (lowest percentage) at the top, to the most popular item (highest percentage) at the bottom."

```
head(myGrocIF,10)

##             baby food  sound storage medium preservation products
##          0.0001016777          0.0001016777          0.0002033554
##       kitchen utensil                  bags        frozen chicken
##          0.0004067107          0.0004067107          0.0006100661
##        baby cosmetics        toilet cleaner        salad dressing
##          0.0006100661          0.0007117438          0.0008134215
##                 whisky
##          0.0008134215

tail(myGrocIF,10)

##          sausage    shopping bags   tropical fruit  root vegetables
##       0.09395018       0.09852567       0.10493137       0.10899847
##    bottled water           yogurt             soda       rolls/buns
##       0.11052364       0.13950178       0.17437722       0.18393493
## other vegetables       whole milk
##       0.19349263       0.25551601

itemFrequencyPlot(myGroc, topN=20)
```




```
ct <- crossTable(myGroc, sort=TRUE)
ct[1:6, 1:6]
```

```
##                   whole milk other vegetables rolls/buns soda yogurt
## whole milk              2513              736        557  394    551
## other vegetables         736             1903        419  322    427
## rolls/buns               557              419       1809  377    338
## soda                     394              322        377 1715    269
## yogurt                   551              427        338  269   1372
## bottled water            338              244        238  285    226
##                   bottled water
## whole milk                  338
## other vegetables            244
## rolls/buns                  238
## soda                        285
## yogurt                      226
## bottled water              1087
```

Explanantion:

1. **ct <- crossTable(myGroc, sort=TRUE)**

This code creates a large grid, like a multiplication table, for every single item in the store. On the left side, it lists all the items (Bread, Milk, Eggs, etc.). Across the top, it lists all the items again (Bread, Milk, Eggs, etc.). Inside the table, the computer fills in a number: How many times were the item on the left and the item on the top bought in the same shopping

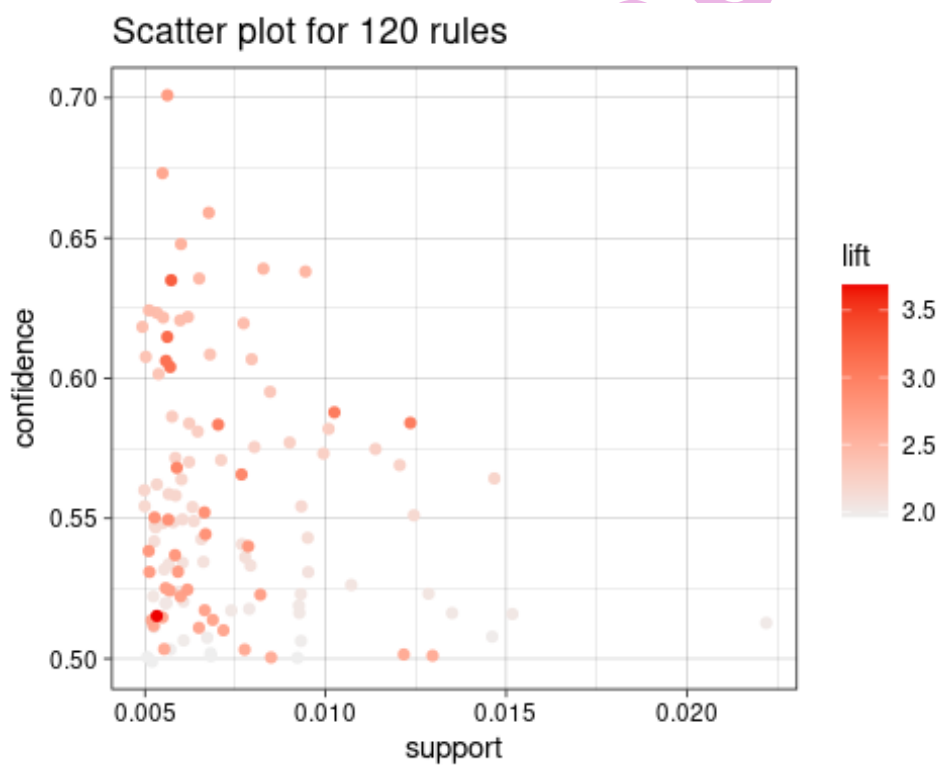
trip? Because of sort=TRUE, the items are organized so the most popular items are shown first.

2. **ct[1:6, 1:6]:** The grid is huge (hundreds of items!). This line says: "Don't show me the whole huge grid. Just show me a small 6x6 box in the top-left corner."

Since the table was sorted, this 6x6 box shows you the relationship between the 6 most popular items in the store.

Model Development

```
#install.packages("arulesViz")

library(arulesViz)

# Now run a test model to examine some rules
ruleset <- apriori(myGroc,
        parameter=list(supp=0.005, conf=0.50),
        control=list(verbose=F))
plot(ruleset)

## To reduce overplotting, jitter is added! Use jitter = 0 to prevent jitter.
```



Explanation:

This command tells the computer to find strong, common buying rules in your grocery data (myGroc).

It's searching for patterns like: "If a customer buys item A, they are also very likely to buy item B."

The code uses the **Apriori algorithm** to scan all your receipts and find item combinations that meet specific minimum requirements:

- **apriori(myGroc, ...):** This starts the search for rules using your grocery transaction data (myGroc).
- **parameter=list(supp=0.005, conf=0.50):** These are the standards a rule must meet to be saved:

  supp=0.005 (Support): The item combination must show up in at least 0.5% of all transactions. This ensures the rules are based on combinations that happen frequently enough to be worth tracking.

  conf=0.50 (Confidence): When the first item(s) in the rule is bought, the second item(s) must be bought with them at least 50% of the time.
- **ruleset <-** ...: All the strong, reliable rules that pass the 0.5% support and 50% confidence tests are saved into a collection called ruleset.

```
highLift <- ruleset[quality(ruleset)$lift > 2.249]
inspectDT(highLift)
```

Explanation:

This code is simply asking the computer to filter the list of all the buying rules you found and show you only the most surprising and strongest ones.

1. **highLift <- ruleset[quality(ruleset)$lift > 2.249]**

- ruleset: This is the big collection of all the items buying rules the computer found in your grocery data
- $lift: Lift is the key idea here. Think of it as the surprise factor of a rule. A lift value of 1 means the two items are bought together exactly as often as you'd expect, given how popular they are individually (no surprise).A lift value greater than 1 means the two items are bought together more often than expected (a pleasant surprise for a store owner!).
- > 2.249: This sets a very high standard. You are only keeping rules where the surprise factor is greater than 2.249. This filters out common-sense rules and keeps only the most unusual and strongest pairings.
- highLift <- ...: The small, filtered list of these top rules is saved into a new variable called highLift.

Preeti Jagadev

2. **inspectDT(highLift)inspectDT(...):** This tells the computer to display the contents of the new list (highLift) in an easy-to-read, interactive table.

Preeti Jagadev